

# AirBnb Project Guide By Aman

Github Url :- <https://github.com/Amanshakya-code/AirBnb>

Please clone this code and just go through code once and prepare this document then you are ready with this project you can include this project in your resume

This project is hard level cover almost every topic that companies asked from the candidate.

## 2. Project Summary (The "What" and "Why")

**Project Title:** Airbnb Backend API **Goal:** To build a scalable backend for booking hotels with dynamic inventory and pricing management.

### Modules & Logic Breakdown:

#### A. Authentication & Security

- **What:** You implemented a dual-token system (Access & Refresh tokens).
- **Why:** Access tokens are short-lived for security; Refresh tokens allow users to stay logged in without re-entering credentials.
- **Key Files:** `AuthController` , `JWTService` , `WebSecurityConfig` .

#### B. Hotel & Room Management

- **What:** CRUD operations for Hotels and Rooms.
- **Why:** Admin (Hotel Managers) need to manage their property details.
- **Key Files:** `HotelController` , `RoomAdminController` .

#### C. Inventory Management (The Core Logic)

- **What:** Instead of just checking if a "Room" exists, you created an `InventoryEntity` which represents a specific room on a specific *date*.
- **Why:** This allows you to manage availability and price **per day**. (e.g., A room might be available on Monday but booked on Tuesday).

#### D. The Booking Lifecycle

- **What:** A state-machine approach to bookings.
  1. **Reserve:** User selects dates -> System locks inventory -> Status: `RESERVED` .
  2. **Payment:** User pays via Stripe -> Status: `PAYMENTS_PENDING` .
  3. **Confirm:** Stripe Webhook confirms success -> Status: `CONFIRMED` .

- **Why:** Prevents booking conflicts and ensures inventory is only permanently deducted after successful payment.

#### E. Dynamic Pricing Engine

- **What:** You calculate prices based on multiple factors (Holidays, Occupancy, Urgency).
  - **Why:** To maximize revenue (Yield Management) similar to real-world apps like Uber or Airbnb.
- 

### 3. Main Components (Interview Focus Areas)

The interviewer will likely focus on the most complex parts of your code. Focus on these three areas:

#### I. Handling Concurrency (Double Booking Problem)

- **Location:** `InventoryRepository.java` & `BookingServiceImpl.java`.
- **The Problem:** Two users try to book the same room for the same dates at the exact same millisecond.
- **Your Solution:** You used **Pessimistic Locking** (`@Lock(LockModeType.PESSIMISTIC_WRITE)`).
- **Code Explanation:** When checking availability, the database locks those specific inventory rows. No one else can read/write to them until the transaction finishes.
- **Interview Tip:** Be ready to explain why you chose Pessimistic Locking over Optimistic Locking (Answer: High contention on inventory suggests preventing conflicts is better than handling failures later).

## II. The Strategy Pattern for Pricing

- **Location:** `com.aman.AirBnb.AirBnb.Strategy` package.
- **The Problem:** Pricing logic is complex and changes often (Surge, Holiday, etc.). Writing one giant `if-else` block is unmaintainable.
- **Your Solution:** You defined a `PricingStrategy` interface. You wrapped strategies inside each other (Decorator/Chain style).
  - `BasePricingStrategy` -> `SurgePricingStrategy` -> `OccupancyPricingStrategy`.
- **Interview Tip:** This is a strong display of Object-Oriented Design.

## III. Stripe Integration & Webhooks

- **Location:** `WebhookController.java` & `BookingServiceImpl.java`.
- **The Flow:** You don't confirm the booking immediately. You initiate a Stripe Session, wait for Stripe to call your `/webhook/payment` endpoint, verify the signature (security), and *then* confirm the booking in the DB.
- **Why:** This ensures you don't give away rooms if the payment fails or the user closes the browser window.

#### 4. Mock Interview Questions & Answers

**Q1: How do you handle a situation where two people try to book the same room instantly?**

- **Answer:** "I implemented `Pessimistic Locking` in my `InventoryRepository`. When a booking is initialized, I run a query `findAndLockAvailableInventory` which locks the specific rows in the `Inventory` table for that date range. This prevents dirty reads and ensures transactional integrity."

**Q2: Why did you create a separate Inventory table instead of just using the Room table?**

- **Answer:** "A `Room` entity defines the physical attributes (capacity, amenities). However, availability is temporal. The `Inventory` entity allows me to track the status (booked, reserved, price) of a room for *every single day*. This enables features like surge pricing on weekends without changing the base price of the room."

**Q3: Explain the Dynamic Pricing logic in your application.**

- **Answer:** "I used the **Strategy Design Pattern**. I have an interface `PricingStrategy` and multiple implementations like `HolidayPricingStrategy`, `UrgencyPricingStrategy`, and `OccupancyPricingStrategy`. In `PricingService`, I chain these strategies together to calculate the final cost dynamically based on real-time data."

**Q4: How does your authentication system work?**

- **Answer:** "I use JWTs with Spring Security. When a user logs in, they receive a short-lived Access Token and a long-lived Refresh Token (stored in an `HttpOnly` cookie). The `JWTAuthFilter` intercepts requests to validate the Access Token. If it expires, the `/refresh` endpoint uses the Refresh Token to issue a new one."

**Q5: How do you handle payment failures?**

- **Answer:** "The booking is initially created with a `RESERVED` status. If the Stripe payment fails or is cancelled, the webhook (or a timeout logic) updates the status to `CANCELLED` and releases the inventory back to the pool."

**Q6: I see a `@Scheduled` task in your code. What does it do?**

- **Answer:** "That is the `PricingUpdateService`. It runs hourly to update inventory prices in bulk. It calculates the dynamic price for upcoming dates and updates a `HotelMinPrice` table, which is an optimization table used to make search queries faster."

# Project Flow

## Detailed Project Process & Flow Layout

This section breaks down exactly how data moves through your application, from the moment a user signs up to when a booking is confirmed.

### 1. Authentication Flow (The Entry Point)

- **Step 1: Sign Up:** User sends details to `/auth/signup`. The system creates a `UserEntity` with `ROLE_GUEST`.
- **Step 2: Login:** User sends credentials to `/auth/login`.
  - **Action:** System validates password and generates two tokens: **Access Token** (short-lived) and **Refresh Token** (long-lived).
  - **Storage:** The Refresh Token is stored in an **HttpOnly Cookie** (to prevent XSS attacks), while the Access Token is sent in the response body.
- **Step 3: Access Protected Routes:** The `JWTAuthFilter` intercepts every request, checks the `Authorization` header, and validates the token before letting the request reach the controllers.

## 2. Hotel & Inventory Management Flow (Admin Side)

- **Step 1: Create Hotel:** Admin creates a hotel via `HotelController` . It is created as "Inactive" by default.
- **Step 2: Create Rooms:** Admin adds rooms ( `RoomAdminController` ).
- **Step 3: Inventory Generation (Critical):**
  - When a room is created/activated, `InventoryService.initializeRoomForAYear()` runs.
  - **Logic:** It creates 365 `InventoryEntity` records (one for each day) for that specific room. This is why you can manage pricing for *specific dates* separately from the room's base info.
- **Step 4: Background Pricing Updates:** The `PricingUpdateService` runs every hour (via `@Scheduled` ). It recalculates prices based on your strategies and updates a specialized table called `HotelMinPriceEntity` to make search queries faster.

## 3. The Booking Flow (User Side - The Core Logic)

This is the most complex flow where race conditions occur.

1. **Search:** User searches by City & Dates ( `HotelBrowseController` ).
    - **Optimization:** The system queries `HotelMinPriceRepository` (a lightweight table) instead of the heavy Inventory table to ensure fast search results.
  2. **Initialize Booking ( `/bookings/init` ):**
- **Concurrency Check:** The system calls `inventoryRepository.findAndLockAvailableInventory()` . This applies a **Pessimistic Write Lock** on the inventory rows. No other transaction can touch these rows until this booking process is done.
  - **Logic:** If availability exists, it subtracts the count from `available` and adds it to `reserved` .
  - **State:** Booking is created with status `RESERVED` .

3. **Add Guests:** User adds guest details. Status changes to `GUESTS_ADDED`.
  4. **Initiate Payment:** System creates a Stripe Session ( `CheckoutService` ) and returns the URL. Status changes to `PAYMENTS_PENDING`.
  5. **Payment Confirmation (Webhook):**
    - User pays on Stripe → Stripe calls your `/webhook/payment` endpoint.
    - **Validation:** System verifies the Stripe signature.
    - **Finalization:** The `BookingService` updates status to `CONFIRMED` and permanently deducts the inventory.
- 

### Critical Interview Questions (Deep Dive)

These questions target the specific architectural decisions in your code.

**Q1: "I see you used a separate `HotelMinPriceEntity` table. Why duplicate data instead of just querying the `Inventory` table?"**

- **The Concept:** This is called a **Materialized View** (or Denormalization).
- **Your Answer:** "Querying the `Inventory` table for search is expensive because it contains millions of rows (Every room × 365 days). To optimize read performance, I created `HotelMinPriceEntity`, which stores only the minimum price per hotel per day. The `PricingUpdateService` updates this table hourly. This separates our **Read** load (Search) from our **Write** load (Bookings), significantly improving search latency."

**Q2: "In `InventoryRepository` , you used `@Lock(LockModeType.PESSIMISTIC_WRITE)` . Why not Optimistic Locking?"**

- **The Concept:** Concurrency Control.
- **Your Answer:** "Optimistic locking (using `@Version` ) is better for low-contention systems where conflicts are rare. However, a hotel booking system is high-contention—multiple users might try to book the last room for New Year's Eve simultaneously. If I used Optimistic locking, users would fill out forms only to be told 'failed' at the very end. **Pessimistic Locking** guarantees the resource is reserved *before* they proceed, providing a better user experience and data integrity."

**Q3: "Explain how your Dynamic Pricing Strategy works. How easy is it to add a 'Rainy Day' discount?"**

- **The Concept:** Open/Closed Principle (SOLID).
- **Your Answer:** "I used the **Strategy Pattern** with a decorator approach. I have an interface `PricingStrategy` and a `PricingService` that chains strategies like `Urgency` , `Occupancy` , and `Holiday` . To add a 'Rainy Day' discount, I would simply create a new class implementing `PricingStrategy` , inject the weather API, and add it to the chain in `PricingService` . I don't need to modify the existing pricing logic, adhering to the Open/Closed Principle."

**Q4: "How does your system handle 'Abandoned Bookings' (Users who reserve but don't pay)?"**

- **The Concept:** Resource Cleanup / State Management.
- **Your Answer:** "When a booking is initialized, the inventory is 'reserved' but not 'booked'. In `BookingServiceImpl` , I have a check `hasBookingExpired` that invalidates bookings older than 10 minutes. In a production environment, I would implement a scheduled job (Cron) that scans for `RESERVED` bookings older than 10 minutes and releases their inventory back to the pool to prevent dead inventory."

**Q5: "Why did you implement `UserDetailsService` in `UserServiceImpl` ?"**

- **The Concept:** Spring Security Integration.
- **Your Answer:** "Spring Security needs a way to load user details from the database during authentication. By implementing `UserDetailsService`, I override the `loadUserByUsername` method to fetch my custom `UserEntity` (via `UserRepository`) and adapt it to the `UserDetails` interface that Spring Security understands. This bridges my custom database schema with Spring's internal security mechanisms."

**Q6: "You are using `BigDecimal` for prices. Why not `Double` ?"**

- **The Concept:** Floating Point Arithmetic Precision.
- **Your Answer:** "Floating-point types like `Double` can introduce rounding errors (e.g.,  $0.1 + 0.2 \neq 0.3$ ) which is unacceptable for financial data. `BigDecimal` provides arbitrary-precision arithmetic, ensuring that price calculations, taxes, and discounts are exact."

## 1. Core Spring Boot & Architecture

**Q1: What is the difference between `@Controller` and `@RestController` ? Which one did you use and why?**

- **Context:** You used `@RestController` in all your controllers (e.g., `HotelController`, `AuthController`).
- **Answer:** "`@Controller` is used for traditional MVC applications where you return a View (like JSP or Thymeleaf). `@RestController` is a convenience annotation that combines `@Controller` and `@ResponseBody`. I used `@RestController` because my application is a RESTful API that returns JSON data directly to the client, not HTML views."

**Q2: How does Dependency Injection work in your project? I see you are using `@RequiredArgsConstructor`.**

- **Context:** Used in almost every service and controller (e.g., `UserServiceImpl`).
- **Answer:** "I am using **Constructor-based Dependency Injection**, which is the recommended approach in Spring. The `@RequiredArgsConstructor` annotation from Lombok generates a constructor for all `final` fields. When Spring initializes the bean, it sees this constructor and automatically injects the required dependencies (like `UserRepository` into `UserServiceImpl`) without needing an explicit `@Autowired` annotation on the constructor."

### Q3: How does Spring Boot handle configuration in your project?

- **Context:** `application.properties` and classes like `StripeConfig`.
  - **Answer:** "I use `application.properties` to define external configurations like Database URLs, JWT secrets, and Stripe keys. Spring Boot automatically loads these. For 3rd party libraries, I created `@Configuration` classes (like `StripeConfig`) where I inject these values using the `@Value` annotation to set up the beans."
- 

## 2. Spring Data JPA & Database

### Q4: You used `@Transactional` in your services. How does it work, and what happens if an exception is thrown?

- **Context:** `BookingServiceImpl.initialiseBooking`, `InventoryServiceImpl.updateInventory`.
- **Answer:** "`@Transactional` wraps the method execution in a database transaction. If the method completes successfully, the transaction is committed. If a `RuntimeException` (unchecked exception) is thrown, Spring automatically **rolls back** the transaction, reverting any changes made to the database within that method. This ensures data consistency, especially in my booking flow where I lock inventory and create a booking record simultaneously."

### Q5: What is the N+1 problem, and how did you avoid it? (Or did you?)

- **Context:** In `HotelEntity`, you have `@OneToMany(fetch = FetchType.LAZY)`.
- **Answer:** "The N+1 problem happens when fetching a parent entity (Hotel) causes N separate queries to fetch its children (Rooms). I used `FetchType.LAZY` for the `rooms` list in `HotelEntity` to prevent loading them unnecessarily. If I need to fetch them together, I would use a `JOIN FETCH` in a custom JPQL query in the repository to load everything in a single query."

**Q6: Explain the `@Query` annotation in `InventoryRepository`. Why did you use it?**

- **Context:** `findHotelsWithAvailableInventory` query.
  - **Answer:** "I used `@Query` to write custom JPQL (Java Persistence Query Language). Standard JPA methods like `findByCity` weren't enough because I needed complex logic: checking availability across a date range, ensuring the room isn't closed, and grouping results. JPQL allowed me to select specific fields and construct a `HotelPriceDTO` directly from the database query."
- 

### 3. Spring Security & JWT

**Q7: Why did you set `SessionCreationPolicy.STATELESS` in `WebSecurityConfig`?**

- **Context:** `securityFilterChain` method.
- **Answer:** "Since I am using **JWT (JSON Web Tokens)** for authentication, the server does not need to store user sessions in memory (like `JSESSIONID`). The token itself contains all necessary user info. Setting the policy to `STATELESS` ensures Spring Security doesn't create or look for an HTTP session, making the application more scalable and truly RESTful."

**Q8: How exactly does your `JWTAuthFilter` intercept requests?**

- **Context:** `JWTAuthFilter` extends `OncePerRequestFilter`.
  - **Answer:** "My `JWTAuthFilter` extends `OncePerRequestFilter`, ensuring it executes once per API request. It checks the `Authorization` header for a 'Bearer' token. If found, it validates the token using `JWTService`. If valid, it extracts the user details and manually sets the `Authentication` object in the `SecurityContextHolder`. This tells Spring Security who the current user is for the duration of that request."
-

#### 4. Advanced Spring Features

**Q9: I see a `GlobalResponseHandler` implementing `ResponseBodyAdvice`. What does this do?**

- **Context:** `GlobalResponseHandler.java`.
- **Answer:** "This is an AOP (Aspect Oriented Programming) concept. It intercepts the response body *before* it is written to the client. I use it to wrap every successful response in a standardized `ApiResponse` object (adding timestamps and status checks) so that the frontend always receives a consistent JSON structure, regardless of which controller method returns the data."

**Q10: How does the `@Scheduled` annotation in `PricingUpdateService` work?**

- **Context:** `updatePrices` method.
- **Answer:** "I enabled scheduling with `@EnableScheduling` in the main class. The `@Scheduled(cron = ...)` annotation tells Spring to run that method automatically based on the cron expression (e.g., every hour). By default, Spring uses a single-threaded task scheduler, so this method runs on a background thread to update hotel prices without user interaction."

**Q11: How did you handle exceptions globally?**

- **Context:** `GlobalExceptionHandler` with `@RestControllerAdvice`.
- **Answer:** "Instead of using try-catch blocks in every controller, I used `@RestControllerAdvice`. It acts as a global interceptor for exceptions thrown anywhere in the controller layer. I defined methods with `@ExceptionHandler` for specific exceptions like `ResourceNotFoundException` or `AccessDeniedException` to return clean, standardized error responses ( `ApiError` ) with the appropriate HTTP status codes."

#### Summary of Spring Annotations You Must Know

Be prepared to explain these in one sentence each:

- `@SpringBootApplication` : Main configuration class; enables auto-config and component scanning.
- `@Service` : Marks a class as a service layer component (business logic).
- `@Repository` : Marks a class as a Data Access Object (database interaction).
- `@Bean` : Manually creates a Spring bean (used in your `SecurityConfig`).
- `@Value` : Injects values from `application.properties`.
- `@Transactional` : Defines the scope of a database transaction.